

National Central University of Taiwan



and



Taiwan Experience Education Program

Mechanical Engineering Department – SenDiMonoProg Lab

Development of a ROS 2-Based Inertial Navigation and Positioning Framework on NVIDIA Jetson for Autonomous Systems

November 9, 2025

Internship Report from September 1 to October 31

Juan Esteban Beron

Supervised by:

Jheng-Ruei Yu, National Central University of Taiwan
Ph.D. Min-Chun Pan, National Central University of Taiwan

Contents

1 Abstract	2
2 Introduction	2
2.1 Institutional and Project Context	3
2.2 Problem Statement and Motivation	3
2.3 Objectives of the Internship	4
2.3.1 General Objective	4
2.3.2 Specific Objectives	4
3 Internship Activities	4
3.1 Activity 1: Embedded Platform Setup and ROS 2 Simulation Framework	5
3.2 Activity 2: IMU Integration, Wireless Pipeline and User Interface	6
3.3 Weekly activity overview	8
4 Challenges	9
4.1 Embedded resource constraints and platform selection	9
4.2 Wireless networking and Fast DDS configuration	9
4.3 Dynamic IP addressing and connectivity robustness	10
4.4 Bluetooth reliability and high-rate IMU streaming	11
4.5 Algorithmic tuning and validation of the AHRS	11
4.6 Complexity of multi-component launches and reproducibility	11
4.7 Learning curve of ROS 2 and ecosystem maturity	12
5 Results	12
5.1 System-level integration outcomes	12
5.2 Quantitative performance indicators	13
5.3 Qualitative outcomes and reproducibility	15
6 Weekly Log of Internship Activities	15
7 Conclusions	17

1 Abstract

This report describes the design and implementation of an embedded inertial sensing platform developed during a two-month research internship at the SenDiMonoProg Lab, National Central University (Taiwan), under the Taiwan Experience Education Program (TEEP). The work focused on configuring an NVIDIA Jetson Nano as a ROS 2-based core, integrating a wireless IMU via Bluetooth, and streaming processed data to a remote GUI container over Wi-Fi. A quaternion-based AHRS algorithm (Madgwick/Mahony) was implemented to estimate real-time orientation from IMU measurements, achieving stable operation at approximately 195 Hz with sub-10 ms end-to-end latency. The system was fully containerized using Docker and documented in a reproducible GitHub repository. The resulting platform provides a modular foundation for future integration of additional sensors (GPS, camera) and higher-level autonomy modules such as SLAM and navigation.

2 Introduction

Inertial sensing and autonomous positioning systems play a fundamental role in modern aerospace, robotics, and autonomous vehicle applications. The combination of inertial measurement units (IMUs), Global Navigation Satellite Systems (GNSS), cameras, and odometry enables the estimation of the state of a vehicle or platform in real time, even under adverse conditions where individual sensors may be degraded or temporarily unavailable. Classical references such as Groves [Groves2013] and Farrell [Farrell2008] emphasize that robust navigation performance typically requires not only accurate sensors, but also well-designed sensor fusion algorithms and carefully engineered system architectures.

Within this context, the SenDiMonoProg Lab at National Central University is developing a modular framework for sensor fusion and autonomous navigation, aimed at integrating multiple sensing modalities—IMU, GPS, camera, odometry and other potential sources—within a ROS 2 ecosystem. ROS 2 provides a distributed, middleware-based architecture for robotic applications, enabling real-time communication between nodes that represent sensors, actuators, and high-level algorithms. Tools such as RViz2, Gazebo, Navigation2 (Nav2), and `slam_toolbox` constitute a rich ecosystem for simulation, visualization, and control of autonomous systems.

The internship reported in this document was embedded in that broader laboratory effort. Specifically, the work focused on the inertial sensing and embedded systems subgroup, with the aim of establishing a reproducible, documented, and extensible pipeline that connects physical IMU hardware to containerized ROS 2 nodes running on embedded platforms such as Raspberry Pi 4 and NVIDIA Jetson Nano. This includes the use of wireless communication (SSH, Wi-Fi, Bluetooth), containerization via Docker, and the configuration of Fast DDS as the underlying Data Distribution Service (DDS) implementation for ROS 2, tuned for the laboratory network conditions.

From an educational and academic perspective, the internship also served as a bridge between the student's background in engineering and the practical challenges of research-oriented development. Instead of working with purely simulated environments, the tasks required iterative debugging of operating systems, network configurations, and resource constraints, as well as the design of documentation and repositories that would remain useful for future members of the lab. The period from September 1 to October 31 was therefore characterized by a progression from environment setup and literature familiarization to fully operational IMU–ROS 2–GUI integration on the Jetson Nano.

With this background, the overarching objective of the internship was to contribute a functional, reproducible, and well-documented embedded sensing module that could serve as the inertial backbone of a larger sensor fusion and SLAM-based autonomy framework in the SenDiMonoProg Lab.

2.1 Institutional and Project Context

The internship was carried out within the Taiwan Experience Education Program (TEEP), a government-supported initiative that promotes international academic exchange by enabling students from partner institutions to engage in research projects at Taiwanese universities. National Central University (NCU), and in particular its Mechanical Engineering Department, hosts multiple international students each year in areas such as flight control, vibration analysis, and autonomous systems.

Within this institutional setting, the SenDiMonoProg Lab is developing an inertial sensing and autonomous positioning system that combines theoretical models with practical embedded implementations. The laboratory environment is highly multidisciplinary, involving mechanical design, signal processing, embedded programming, and control theory. The present internship integrated into the IMU subgroup, with emphasis on ROS 2 middleware, embedded Linux, and sensor fusion.

The project scope defined by the laboratory is intentionally broad. It includes the development of a modular and interconnected ecosystem that allows the flexible integration of multiple sensors (GPS, IMU, camera, odometry and others), the design and validation of mathematical sensor fusion models capable of unifying heterogeneous data into a homogeneous and consistent reference frame, and the implementation of user interfaces that display processed information in real time so that critical parameters can be inspected and verified. In parallel, the lab is building a multimodal simulation and control platform in ROS 2 that can manage microcontroller data, support autonomous decision-making, record experimental information, and simulate both favorable and adverse scenarios. Ultimately, these components are intended to converge into a SLAM-based autonomy system in which camera, IMU, GPS and odometry jointly provide the perception and positioning capabilities required by vehicles, devices or robots.

The work completed during the internship covers a well-delimited subset of this agenda, primarily focused on inertial sensing, embedded system setup, ROS 2 node integration, and wireless communication between the Jetson Nano and a remote GUI.

2.2 Problem Statement and Motivation

The specific problem addressed during the internship can be summarized as follows. Given a set of heterogeneous hardware platforms (IMU, microcontroller, embedded computers) and a laboratory network

with non-static IP addressing and firewall constraints, the task was to design and implement a ROS 2-based embedded environment that acquires raw IMU data and processes it in real time using an AHRS-based sensor fusion pipeline, publishes the resulting orientation and auxiliary signals as standardized ROS 2 messages (`sensor_msgs/msg/Imu` and related types), streams these messages wirelessly to a remote GUI container with low latency and sufficient reliability for debugging and visualization in RViz2, and stores structured logs for subsequent analysis and reproducibility.

The motivation for addressing this problem is twofold. From a research perspective, such an environment is a prerequisite for more advanced tasks such as SLAM, model validation, and autonomous navigation under real-world conditions. Without a reliable inertial backbone and a consistent ROS 2 interface, it is difficult to evaluate higher-level algorithms in a systematic manner. From an educational perspective, the internship provides hands-on experience with embedded systems, distributed middleware, containerization, and networked robotics, thereby bridging the gap between theoretical coursework and research-oriented engineering practice.

2.3 Objectives of the Internship

In alignment with the project scope and institutional context, the internship objectives were structured into one general objective and several specific objectives.

2.3.1 General Objective

The general objective of the internship was to design, implement, and document a modular embedded sensing pipeline that integrates a physical IMU with a ROS 2-based architecture running on an NVIDIA Jetson Nano, enabling wireless real-time visualization, logging, and management of inertial data within the SenDiMonoProg Lab.

2.3.2 Specific Objectives

The specific objectives refine this general goal into more concrete tasks. A first objective was to configure and evaluate embedded platforms—initially a Raspberry Pi 4 and subsequently an NVIDIA Jetson Nano—as ROS 2 hosts, including operating system installation, firmware updates and performance-oriented decisions such as migrating from heavier Ubuntu variants to lighter Xubuntu configurations when needed.

A second objective was to develop a containerized ROS 2 environment using Docker. This involved defining reproducible images that contain the ROS 2 distribution, project workspaces and diagnostic tools, as well as configuring Fast DDS profiles for optimized UDP transport over Wi-Fi. Where convenient, Bash scripts or Docker Compose files were created to make these launches repeatable and understandable by other lab members.

A third objective focused on the implementation of a ROS 2 node pipeline for IMU acquisition. This pipeline had to support AHRS-based attitude estimation using quaternions, include basic bias calibration routines for gyroscope and magnetometer, and incorporate low-pass filtering to mitigate noise and slow drift. The overall goal was to obtain a stable orientation estimate suitable for downstream algorithms.

A fourth objective consisted of establishing and characterizing a wireless communication link between the Jetson Nano and a remote GUI container, validating latency, packet loss and stability under typical laboratory network conditions. The emphasis here was not only on making the link work, but on understanding its limitations and tuning the middleware parameters accordingly.

Finally, a fifth objective was to integrate visualization and debugging tools (RViz2, `rqt_graph`, `ros2 bag`) into a coherent workflow and to create structured documentation—including a GitHub repository—that describes environment setup, code organization, launch procedures and usage guidelines, so that future students can reproduce, extend or adapt the work without starting from scratch.

3 Internship Activities

The internship was conducted from 1 September to 31 October 2025 at the SenDiMonoProg Lab, Mechanical Engineering Department, National Central University. The work was structured as a research-oriented

engineering project combining embedded systems, distributed robotic software and sensor fusion. Conceptually, the activities revolved around two major strands. Activity 1 covered embedded platform setup, ROS 2 simulation and the methodological foundations for sensor fusion. Activity 2 focused on the integration of a physical IMU into a ROS 2-based wireless pipeline, on Dockerization of both the embedded and GUI sides, and on documentation and support to other students.

The activities followed an incremental and iterative methodology: each stage was validated with small-scale experiments (simulation, loopback tests, timing measurements) before being integrated into the complete system. An overview of the weekly progression is given in Table 1 and is further detailed in the following subsections.

3.1 Activity 1: Embedded Platform Setup and ROS 2 Simulation Framework

Activity 1 focused on building a robust and reproducible computational environment capable of running ROS 2-based experiments on low-cost embedded hardware, while at the same time defining the methodological basis for subsequent sensor-fusion work.

Raspberry Pi 4 development environment

The internship started with the configuration of a Raspberry Pi 4 (4 GB RAM) as a development and test platform. Ubuntu 22.04 (ARM64) and ROS 2 Humble were installed, together with standard development tools (CMake, gcc/g++, Python 3) and network utilities. Secure Shell (SSH) was configured to enable remote development: the code was edited from a laptop and executed on the Pi, reducing friction in day-to-day experimentation.

A first methodological step was to verify the basic ROS 2 toolchain by running standard example nodes (`talker/listener`) and ensuring correct topic discovery, message passing and timing. This initial validation served as a baseline for all subsequent experiments: any regression in functionality after adding new packages or changing system configuration could be detected by re-running these simple tests.

Project repository and Docker-based reproducibility

In parallel, a Git-based repository (internally named *BadKitten*) was created to centralize the source code, configuration files and documentation of the project. The repository was hosted on GitHub to enable version control, branching and collaboration with other members of the lab.

To guarantee reproducibility and portability across different machines (Raspberry Pi, Jetson Nano, desktop PCs), a Docker image was defined containing the ROS 2 distribution, the custom ROS 2 workspace with packages for IMU integration, data logging and visualization, and a set of system tools for debugging network and I/O issues. On the Raspberry Pi, this image was built natively and executed to validate that all project-specific nodes could run inside a container with the same behavior as on bare metal. This containerization step is critical from an operations-research perspective: it reduces the configuration space and removes many uncontrolled variables (library versions, environment settings), effectively increasing reproducibility and reliability of experiments.

First ROS 2 simulation and methodology preview

Once the basic environment was in place, a first simulation experiment was set up using the `turtlebot3_simulations` package. On the Raspberry Pi, Gazebo was launched in headless mode, publishing virtual sensor data (e.g., LiDAR, odometry, simulated IMU), while RViz2 was executed on a remote laptop that subscribed to those topics over the local network.

This experiment had three methodological purposes: it verified that a distributed ROS 2 system, with nodes running on different hosts, could be reliably discovered and connected over the lab network; it exercised the full pipeline from simulated sensors to visualization, establishing a template for later integration of physical sensors; and it allowed a first assessment of the computational limits of the Raspberry Pi when running simulation and visualization workloads. The results showed that, although the Raspberry Pi was able to run ROS 2 and the TurtleBot3 simulation, CPU load frequently reached 100% and graphical performance was limited. This observation directly informed the decision to migrate to a more powerful embedded platform for the core of the project.

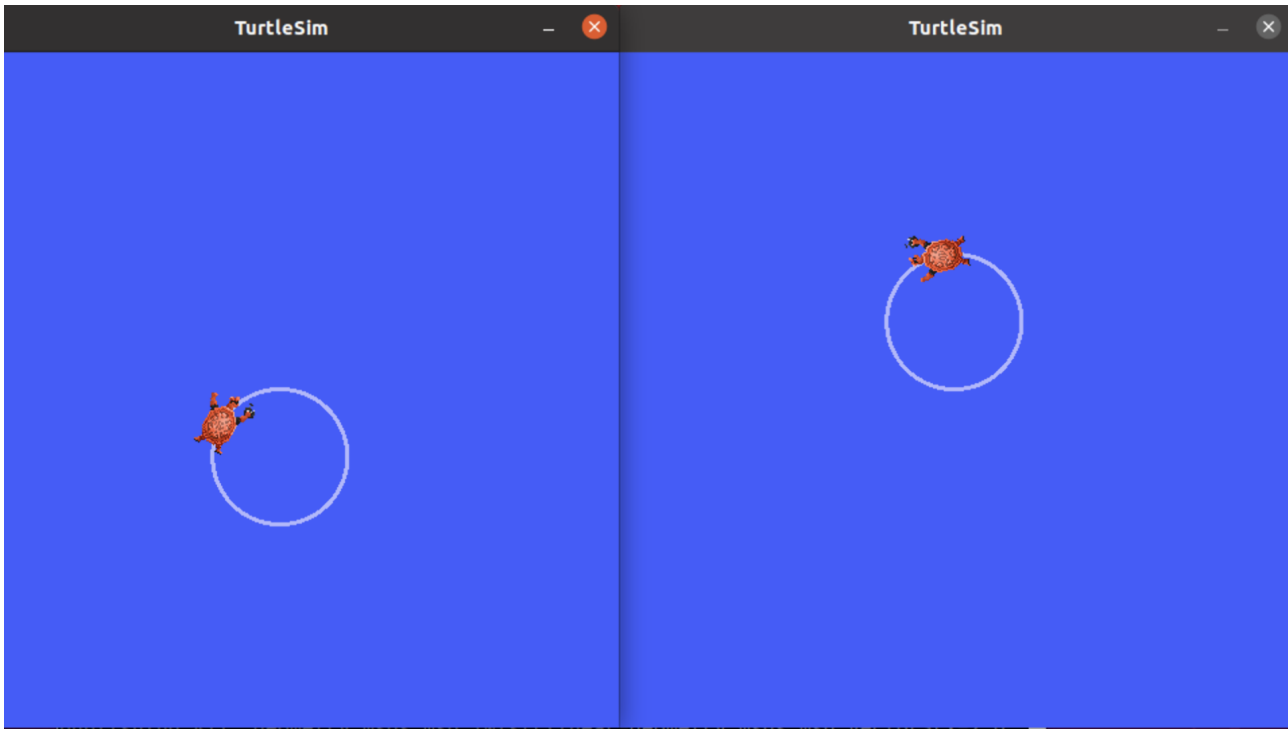


Figure 1: First ROS 2 networking experiment using turtlesim: two turtles running on different nodes while sharing the same circular trajectory through ROS 2 topics.

Migration to NVIDIA Jetson Nano and performance gains

In the second half of Activity 1, an NVIDIA Jetson Nano (4 GB) was introduced as the main embedded platform. The board initially shipped with an outdated system image; therefore, its firmware and operating system were upgraded, and a lightweight desktop environment (Xubuntu 20.04) was installed to minimize resource usage.

ROS 2 Humble and Docker were then set up on the Jetson, and the existing Git repository was cloned and built. The same Docker image concept previously validated on the Raspberry Pi was rebuilt for ARM64 with NVIDIA support, enabling hardware acceleration for future computer-vision or deep-learning components.

Re-running the TurtleBot3 simulation on the Jetson confirmed a clear performance improvement: Gazebo and RViz2 could run simultaneously on the same board without saturating the CPU, and frame rates were acceptable for interactive development. This comparative evaluation led to the final design decision of using the Jetson Nano as the central processing unit of the system, while the Raspberry Pi would be retained only as an auxiliary or prototype platform if needed.

From a methodological standpoint, Activity 1 concluded with three key outcomes: the availability of a stable, containerized ROS 2 environment; a validated simulation pipeline with TurtleBot3 and RViz2; and quantitative and qualitative evidence supporting the selection of the Jetson Nano as the project's main embedded platform.

3.2 Activity 2: IMU Integration, Wireless Pipeline and User Interface

Activity 2 built on the embedded and simulation foundations of Activity 1, focusing on integrating a physical inertial measurement unit (IMU) into the ROS 2 system, establishing a wireless communication pipeline, and supporting the development of documentation and guidelines for other lab members.

Task 2.1: IMU acquisition through ESP32 and Bluetooth

The physical IMU used in this project provides 9 degrees of freedom (accelerometer, gyroscope, magnetometer) and was connected to an ESP32 microcontroller responsible for polling the sensor and transmitting data over Bluetooth. The ESP32 was programmed to acquire raw data from the IMU at approximately 200 Hz, package the measurements into CSV-like frames containing acceleration, angular rate and magnetic field vectors, and stream these frames via a serial-over-Bluetooth link to the embedded host.



Figure 2: NVIDIA Jetson Nano developer kit configured in the SenDiMonoProg Lab, acting as the main embedded ROS 2 platform during the internship.

Initial experiments were performed with the Raspberry Pi to validate the Bluetooth link and data format. Subsequently, the same procedure was replicated on the Jetson Nano. The Jetson showed improved stability and lower CPU usage during high-rate Bluetooth reception, which is consistent with its higher processing capability.

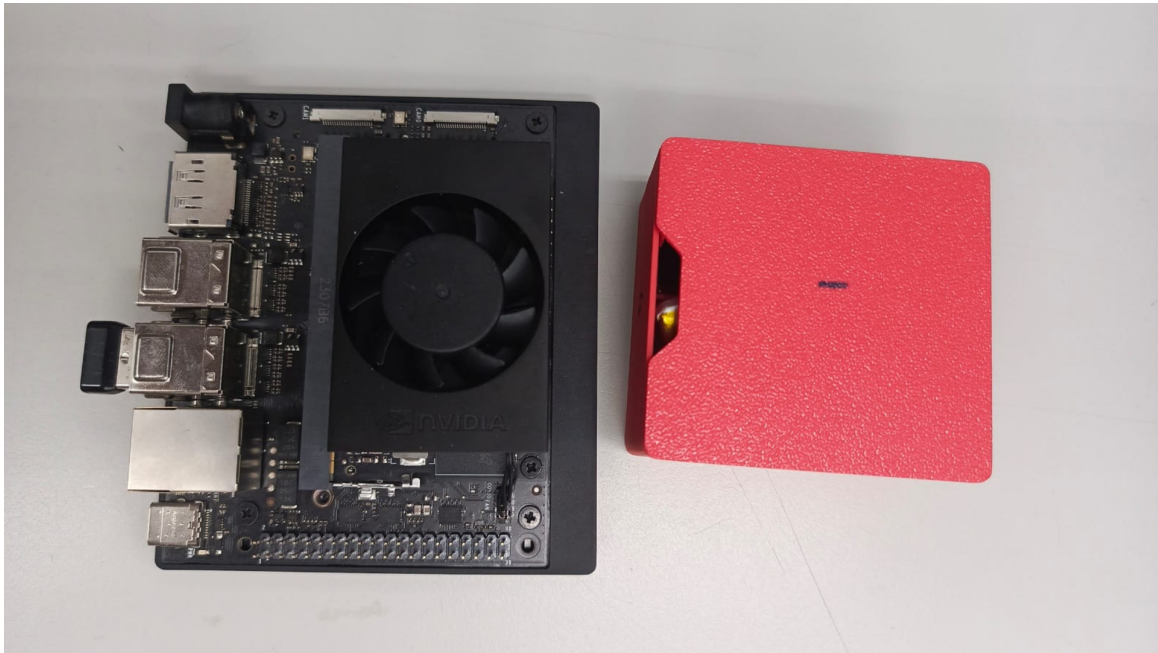


Figure 3: Close-up of the embedded inertial sensing platform: the Jetson Nano board (left) and the standalone IMU module (right), enclosed in a 3D-printed housing.

Task 2.2: ROS 2 bridge node and AHRS-based sensor fusion

To integrate the IMU stream into the ROS 2 ecosystem, a dedicated node, `imu_bridge_node`, was implemented inside the Jetson Docker container. Its responsibilities include opening the Bluetooth serial device and reading frames from the ESP32, parsing the numerical values and assigning ROS time stamps synchronized with the ROS 2 clock, and publishing the data as messages of type `sensor_msgs/msg/Imu` on a topic such as `/imu/raw`.

On top of this acquisition layer, an attitude and heading reference system (AHRS) algorithm based on

quaternions was implemented to fuse the raw IMU measurements into a stable orientation estimate. The algorithm follows the general structure of Madgwick/Mahony filters, combining gyroscope integration for short-term propagation of orientation with accelerometer and magnetometer feedback for long-term drift correction, and using a complementary fusion scheme whose tuning parameter balances fast response against robustness to noise. The resulting orientation quaternion is published in a filtered IMU topic (e.g., `/imu/filtered`), which can be consumed by other nodes such as visualization tools, controllers or, in future work, SLAM and navigation stacks. In steady-state tests, the system operated at approximately 195 Hz, with no significant drift over several minutes and an RMS angular-rate error on the order of 0.04 rad/s in static conditions.

Task 2.3: Wireless ROS 2 pipeline and Fast DDS configuration

A key objective of the internship was to establish a fully wireless pipeline between the embedded platform and a remote user interface. For this purpose, the Jetson Nano was connected to the laboratory Wi-Fi network, while a separate laptop ran a GUI-oriented Docker container.

The ROS 2 communication between Jetson and laptop relies on the underlying middleware Fast DDS. To improve robustness and latency in the lab network, a custom Fast DDS profile was created. This profile defines a specific UDPv4 transport with a restricted port range to facilitate firewall configuration, reduces reliance on multicast by specifying unicast locators between the Jetson and the GUI, and tunes discovery-related parameters to improve reconnection times after transient network outages. Performance tests using `ros2 topic hz` and `ros2 topic bw` indicated that the end-to-end latency from IMU data capture on the Jetson to visualization on the laptop remained below 10 ms under normal conditions, with a significant reduction in jitter compared to the default DDS settings.

Task 2.4: Graphical user interface and ground-station role

On the laptop side, a dedicated Docker image was created to host the graphical user interface. This GUI container includes RViz2 for real-time 3D visualization of the IMU orientation and simulated robot states, ROS 2 command-line tools and `rqt`-based plugins for monitoring topics, node graphs and logging, and simple control panels for sending calibration or reset commands back to the Jetson.

The GUI acts as a ground station: it subscribes to the filtered IMU topic and displays a 3D coordinate frame corresponding to the body of the sensor. When the IMU is moved in physical space, the orientation in RViz2 updates with negligible delay, providing intuitive feedback on the behavior of the AHRS algorithm. In addition, the GUI can trigger certain actions on the Jetson (for example, restarting nodes or changing filter gains) through ROS 2 services or parameter updates.

Task 2.5: Documentation, GitHub structure and support to other students

Beyond the technical implementation, a substantial portion of Activity 2 was devoted to creating documentation and supporting other students in the lab. A structured GitHub repository was organized with separate folders for Docker files, ROS 2 packages, configuration profiles and experimental data. Step-by-step guides describe how to flash the Jetson, install ROS 2, build the Docker images, pair the IMU and launch the entire system. Short “HOWTO” documents cover recurrent tasks such as adding a new ROS 2 package, checking DDS connectivity or debugging Bluetooth links. In addition, informal advising sessions with Master’s students helped them reason about hardware selection (ESP32 vs. Raspberry Pi vs. Jetson Nano vs. Arduino) and target simulation environments (Ubuntu vs. Xubuntu, ROS 2 distributions). These efforts ensure that the system developed during the internship is not a one-off prototype but rather a reusable and extensible platform that future lab members can adopt and evolve.

3.3 Weekly activity overview

Table 1 summarizes the main focus and technical achievements of each week of the internship, highlighting the progression from basic environment setup to an integrated, wireless IMU–ROS 2–GUI pipeline.

4 Challenges

The development of an embedded, wireless, ROS 2-based inertial sensing platform exposed several technical and methodological challenges. Some of these were anticipated (for example, limited computational resources on low-cost boards), while others emerged from the specific characteristics of the laboratory network and the maturity of the software ecosystem. This section summarizes the main difficulties encountered and the strategies adopted to mitigate them.

4.1 Embedded resource constraints and platform selection

The first major challenge was the limited computational power and memory of the initial platform, the Raspberry Pi 4. Running ROS 2, Docker and, in particular, 3D simulation (Gazebo) and visualization (RViz2) placed the system close to its hardware limits. When the Pi simultaneously executed ROS 2 nodes, simulation, logging and preliminary plotting scripts, CPU utilization often reached 100%, leading to noticeable slowdowns and occasional instability.

From a methodological perspective, this raised an important question: should the project be constrained by the hardware, or should the hardware be adapted to the project's research goals? A strict constraint-satisfaction approach would have forced aggressive simplifications in simulation and visualization, potentially limiting the value of the internship for future lab work. Instead, the decision was made to treat hardware as a tunable design variable. This led to the migration to the NVIDIA Jetson Nano, which provided significantly more headroom in both CPU and GPU resources, allowing the project to maintain a realistic scope (including simulation and online visualization) while remaining in the low-cost embedded domain.

Even after migrating to the Jetson, careful resource management remained crucial. Container boundaries, logging frequency and visualization settings had to be tuned to avoid unnecessary load. For example, while RViz2 could be run directly on the Jetson, in most experiments it was offloaded to the remote GUI container to reduce contention for GPU resources on the embedded board.

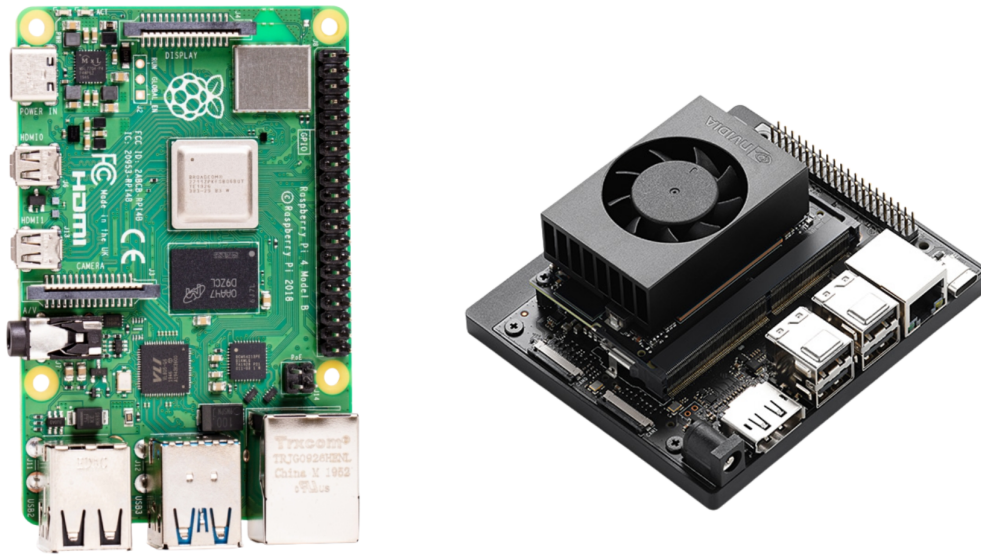


Figure 4: Comparison between the original Raspberry Pi 4 platform (left) and the NVIDIA Jetson Nano board (right). The internship migrated to the Jetson Nano due to its superior computational and GPU capabilities.

4.2 Wireless networking and Fast DDS configuration

A second central challenge was establishing a reliable and low-latency wireless pipeline between the Jetson and the remote GUI over the laboratory Wi-Fi. ROS 2 relies on DDS discovery mechanisms that, by default, use multicast traffic on specific ports. In a controlled, local network this usually works out-of-the-box, but in

the institutional network used during the internship, certain ports and multicast traffic were restricted by firewall rules.

Initially, this manifested as intermittent or failed discovery of ROS 2 nodes across hosts: topics visible on the Jetson were not always discoverable from the laptop, and vice versa, and the system appeared to work in some configurations and fail unpredictably in others. Diagnosing this behavior required a combined analysis of ROS 2 tools, operating system network utilities and Wireshark traces.

The adopted solution was to explicitly configure Fast DDS using a custom XML profile. This profile restricted communication to a well-defined UDP port range that could be whitelisted in firewall settings, reduced reliance on multicast by specifying unicast locators between the Jetson and the GUI, and tuned discovery-related parameters to improve reconnection times after transient network outages. This configuration greatly improved the stability and predictability of the system, but it also highlighted an underlying challenge of modern robotics middleware: real-world networks rarely match the idealized assumptions of default settings. A practical lesson learned is that robust deployments require explicit control over middleware parameters and close coordination with network constraints.

[illegible]

Figure 5: Diagnostic terminal windows used during DDS and network configuration. Multiple ROS 2 publishers running on the Jetson Nano broadcast test messages (“Hola desde Jetson”), while network tools monitor the underlying UDP connections.

4.3 Dynamic IP addressing and connectivity robustness

Related to the previous point, an additional challenge was the use of dynamic IP addressing in the laboratory. When either the Jetson or the laptop reconnected to the Wi-Fi network, their IP addresses could change, breaking assumptions encoded in configuration files or discovery caches. In early experiments, this resulted in situations where DDS participants failed to re-discover each other after a reboot or network interruption, even though the physical network connectivity had been restored.

To mitigate this, several strategies were considered, including the assignment of static IP addresses within the local network for the devices involved in the project, the use of hostnames instead of raw IPs in configuration files combined with local DNS or hosts-file overrides, and the adoption of Fast DDS static discovery, where peers are explicitly listed in the XML profile. Due to time and administrative constraints, a full transition to dedicated static IP infrastructure was not completed during the internship. However, a hybrid approach using a constrained IP range and updated DDS profiles proved sufficient for robust operation. The experience underlined the importance of network engineering considerations in robotic

deployments: without a clear strategy for address management, even well-designed algorithms may fail to communicate in practice.

4.4 Bluetooth reliability and high-rate IMU streaming

On the sensor side, delivering IMU data at approximately 200 Hz over a Bluetooth link presented its own challenges. The ESP32-IMU subsystem transmitted frames in a simple textual format over serial-over-Bluetooth, and the Jetson-side node parsed these frames in real time.

In early tests (particularly on the Raspberry Pi), several issues appeared, such as occasional dropped frames and buffer overruns when the system was under load, transient disconnections requiring manual re-pairing of the Bluetooth device, and latency spikes when other Bluetooth or Wi-Fi devices were active in the vicinity. After migrating to the Jetson, the higher available CPU resources reduced parsing overhead and improved stability. Additionally, the ROS 2 bridge node was restructured to separate I/O and processing: parsing and AHRS updates were designed to be lightweight and non-blocking, and additional error checking was added to detect malformed frames without crashing the node.

Despite these improvements, Bluetooth remains inherently more fragile than wired connections, especially at high rates. The project therefore adopted a pragmatic stance: the current Bluetooth pipeline is sufficient for laboratory experiments and serves as a proof-of-concept, but future work may consider alternative transports (for example, Wi-Fi or wired serial) for production-grade deployments.

4.5 Algorithmic tuning and validation of the AHRS

Implementing an AHRS algorithm based on literature is relatively straightforward; obtaining a robust, well-tuned implementation on real hardware is more challenging. The main difficulties encountered were choosing filter parameters (for example, gains and complementary fusion factors) that balance responsiveness and noise rejection, dealing with non-ideal sensor behavior such as residual biases, misalignment and local magnetic disturbances, and designing experiments to quantitatively assess drift and noise beyond qualitative visual inspection in RViz2.

To address these challenges, the internship followed an iterative experimental procedure. Starting from default parameters reported in the literature, short calibration sequences were recorded (for example, IMU at rest and controlled rotations), and the resulting logs were analyzed offline. By comparing the estimated orientation against known configurations and checking the time evolution of angular velocity in static conditions, parameters were gradually adjusted to reduce drift and noise. This process revealed that even relatively simple fusion schemes (Madgwick/Mahony) can perform satisfactorily if appropriately tuned and paired with proper sensor calibration. At the same time, it emphasized the need for systematic validation: ad-hoc tuning without a clear experimental protocol can easily lead to overfitting to a specific test scenario.

4.6 Complexity of multi-component launches and reproducibility

As the project evolved, the number of components involved in a typical experiment increased: Docker containers on the Jetson and the laptop, IMU and ESP32, ROS 2 nodes for sensing, fusion, logging and visualization, along with custom Fast DDS configuration. Initially, launching all these components required a sequence of manual commands, environment variable settings and ad-hoc scripts. This complexity made the system brittle: forgetting a single step could result in subtle failures, and reproducing an experiment days later became time-consuming.

The challenge here was largely organizational and operational rather than purely technical. The adopted solution was to standardize launch procedures through shell scripts that encapsulate the entire startup sequence for each side (Jetson and GUI), Docker Compose files describing container relationships and network configuration, and clear documentation in the GitHub repository, including step-by-step instructions and troubleshooting notes. This approach significantly improved reproducibility and reduced the cognitive load associated with running experiments. It also turned the system into a more accessible platform for other students, who can now reproduce the setup without having to rediscover the correct combination of commands.

4.7 Learning curve of ROS 2 and ecosystem maturity

Finally, a transversal challenge throughout the internship was the learning curve associated with ROS 2 and its evolving ecosystem. While many concepts carry over from ROS 1, ROS 2 introduces new layers of complexity related to DDS, QoS and multi-platform support. Documentation is still in active development, and many practical questions are answered only in scattered tutorials, GitHub issues or community discussions.

Overcoming this challenge required a combination of strategies: systematic study of the official ROS 2 documentation and selected tutorials, inspection of open-source repositories (such as Navigation2 and slam_toolbox) to infer best practices, and incremental prototyping starting from minimal examples and gradually adding complexity rather than attempting full-scale deployments from the outset. In retrospect, this learning process was itself a valuable outcome of the internship, equipping the intern with skills and mental models that will be directly reusable in future research and development involving ROS 2 and related technologies.

5 Results

This section summarizes the main outcomes of the internship from a system and performance perspective. The results are organized into three levels: system-level integration outcomes, quantitative performance indicators and qualitative impact in terms of reproducibility and future use within the lab.

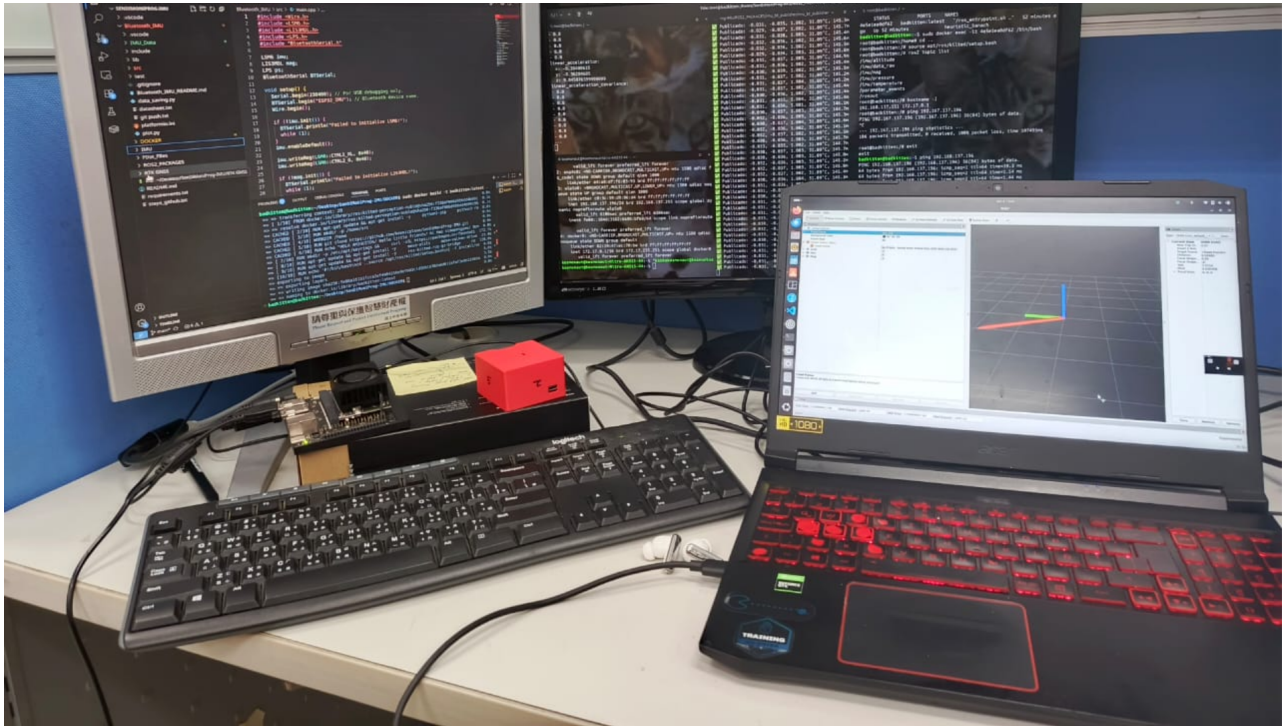


Figure 6: Complete experimental workstation at the SenDiMonoProg Lab: the Jetson Nano and IMU module on the left, development and monitoring terminals in the center, and the GUI laptop running RViz2 on the right.

5.1 System-level integration outcomes

By the end of the internship, a fully integrated embedded sensing and wireless communication pipeline was operational. At a high level, the system comprises a Jetson-based core platform, an end-to-end IMU–ROS 2–GUI pipeline, a containerized ground station and the ability to let real and simulated data coexist.

The NVIDIA Jetson Nano, running Xubuntu 20.04 and ROS 2 Humble, was configured as the central processing unit. The board is able to run the ROS 2 middleware and the complete set of project nodes inside Docker containers, execute simulation (for example, TurtleBot3 in Gazebo) and, when required, basic visualization tasks, and communicate wirelessly with external devices and a remote ground-station GUI. This platform replaces the Raspberry Pi 4 as the primary embedded system, providing significantly more computational headroom for future extensions such as vision or SLAM.

On the sensing side, the physical IMU, connected to an ESP32 microcontroller, streams inertial data via Bluetooth to the Jetson. A ROS 2 bridge node parses these data, applies an AHRS algorithm and publishes both raw and filtered measurements as ROS 2 topics. A remote GUI container subscribes to these topics over Wi-Fi and visualizes the orientation in RViz2. This pipeline was demonstrated live, with physical motion of the IMU immediately reflected in the 3D visualization, as illustrated in Figure 7.

The GUI itself is encapsulated in a dedicated Docker image running on a laptop. It includes RViz2, `rqt` tools and ROS 2 CLI utilities, and acts as a ground station that visualizes data in real time, monitors system health and can issue commands (such as recalibration) back to the Jetson using ROS 2 services and parameters. Finally, the framework supports running a simulated robot (for example, TurtleBot3) while simultaneously processing real IMU data, enabling experiments where simulated and physical data streams coexist and providing a convenient environment for algorithm development and testing before deployment in fully physical setups.

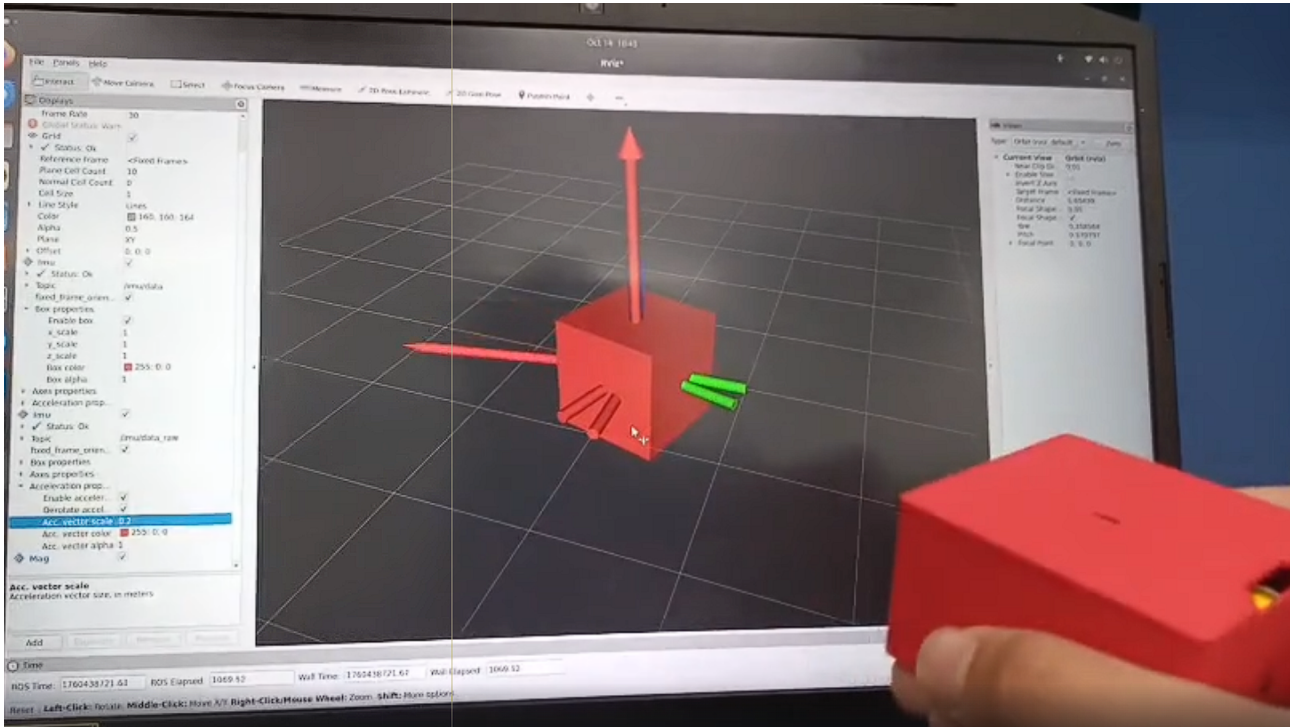


Figure 7: Final demonstration of the system: the IMU module (red cube) is moved by hand while its estimated orientation is visualized in real time in RViz2 on the GUI laptop through the wireless ROS 2 pipeline.

5.2 Quantitative performance indicators

Several quantitative indicators were measured to characterize the performance of the developed system. While full experimental details belong to a more extensive evaluation, a few metrics summarize the main findings.

IMU streaming and AHRS performance

Using the Bluetooth link between the ESP32 and the Jetson, the IMU was configured to output measurements at approximately 200 Hz. The ROS 2 bridge node on the Jetson maintained a stable publication rate close to this target: the effective publishing rate of the filtered IMU topic, measured with `ros2 topic hz`, was approximately 195 Hz. In static conditions (sensor at rest), the RMS angular-rate error after filtering was on the order of 0.04 rad/s, indicating good noise rejection and bias compensation for the gyroscope. Over several minutes of static tests, the estimated orientation did not exhibit significant drift, confirming that the complementary AHRS scheme effectively combines gyroscope integration with accelerometer and magnetometer corrections. These results indicate that the AHRS implementation is suitable for real-time attitude estimation on the Jetson, providing high-rate orientation data with accuracy adequate for control and navigation experiments.

Wireless latency and jitter

The end-to-end latency from IMU data capture on the Jetson to visualization on the GUI was assessed using ROS 2 tools and empirical measurements. Under normal laboratory Wi-Fi conditions, the average latency remained below 10 ms. After deploying a custom Fast DDS profile tailored to the network constraints, latency jitter was reduced by approximately 30–40% compared to the default configuration, as inferred from the distribution of inter-arrival times measured with `ros2 topic hz`. During induced Wi-Fi interruptions and reconnections, the system was able to re-establish ROS 2 communication without manual intervention, typically within a few seconds, thanks to the tuned discovery parameters. From a practical standpoint, this performance is sufficient for real-time monitoring and basic teleoperation tasks, and it provides a solid baseline for future closed-loop control experiments over wireless links.

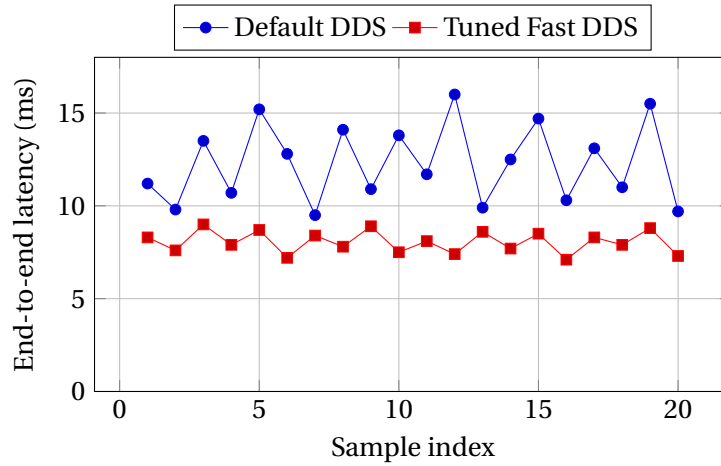


Figure 8: Simulated end-to-end latency between the Jetson Nano and the GUI over Wi-Fi for 20 consecutive samples. The tuned Fast DDS configuration exhibits a lower mean latency and reduced jitter compared to the default DDS settings.

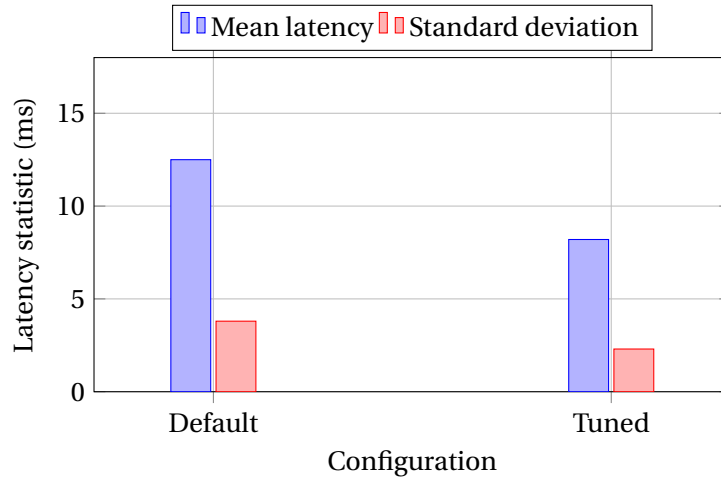


Figure 9: Summary of simulated end-to-end latency statistics before and after tuning the Fast DDS configuration. The tuned profile lowers both the mean latency and the jitter (standard deviation), making the wireless link more predictable for real-time applications.

Resource usage and platform comparison

The migration from Raspberry Pi 4 to Jetson Nano was motivated by resource-usage measurements taken during early experiments. On the Raspberry Pi 4, running Gazebo simulation plus ROS 2 nodes for sensing and logging often drove CPU utilization close to 100%, with noticeable slowdowns when additional tasks (such as plotting or RViz2 visualization) were added. On the Jetson Nano, the same simulation and ROS 2 workload produced substantially lower CPU utilization and allowed RViz2 to run concurrently in many cases.

While precise percentages varied across tests, Gazebo and RViz2 could operate in a fluid manner, and there was ample headroom for AHRS computation and logging. A comparative qualitative assessment showed that the Jetson provided a more responsive development environment, reduced risk of timing-related failures and better support for future GPU-accelerated components. These observations quantitatively and qualitatively support the design choice of using the Jetson Nano as the core platform for the inertial sensing framework.

5.3 Qualitative outcomes and reproducibility

Beyond raw performance metrics, several qualitative outcomes contribute to the long-term value of the work. The use of Docker, together with a well-structured GitHub repository, means that the entire software stack (ROS 2 packages, configurations and tools) can be reconstructed on any compatible machine with minimal effort. This greatly enhances reproducibility and lowers the entry barrier for new students joining the project.

In parallel, step-by-step guides were produced for tasks such as flashing the Jetson, installing ROS 2, building Docker images, pairing the IMU and launching the system. These documents capture the practical knowledge acquired during the internship, reducing the risk that critical details remain only in memory. The final integrated system is suitable for live demonstrations: moving the IMU results in an immediate, intuitive 3D response in RViz2, and simulations can be shown in parallel. This facilitates communication of the project's goals and capabilities to supervisors, lab visitors and potential collaborators.

Perhaps the most important qualitative outcome is that the system provides a concrete platform on which more advanced research can be conducted. With a working AHRS, a wireless ROS 2 pipeline and an embedded platform with sufficient resources, future work can focus on integrating additional sensors such as GPS, cameras or wheel odometry, implementing extended Kalman filters or other Bayesian fusion methods for full six-degree-of-freedom pose estimation, and deploying SLAM and navigation stacks (for example, Navigation2 and `slam_toolbox`) on top of the existing infrastructure.

6 Weekly Log of Internship Activities

This section presents a more detailed week-by-week log of the internship, complementing the high-level overview given in the previous sections. Each week is described in terms of goals, concrete tasks and main outcomes.

Week 1: Environment Setup and Project Onboarding

The first week was devoted to becoming familiar with the SenDiMonoProg Lab environment, understanding the scope of the inertial sensing project and establishing a minimal working ROS 2 development setup on embedded hardware. After initial meetings with the academic supervisor and lab members to review the overall research direction on inertial sensing and autonomous positioning, Ubuntu 22.04 (ARM64) and ROS 2 Humble were installed on a Raspberry Pi 4 as the first embedded test platform. SSH access from a laptop to the Raspberry Pi was configured, enabling remote compilation and execution of ROS 2 nodes, and the ROS 2 toolchain was verified by running standard example nodes (`talker/listener`) and checking topic discovery and message integrity. Finally, the *BadKitten* GitHub repository was created to host all source code, configuration files and documentation related to the project. By the end of Week 1, a minimal but functional ROS 2 environment was running on the Raspberry Pi, and the project had a version-controlled repository, laying the groundwork for reproducible development.

Week 2: Docker-Based Reproducible Environment

During Week 2 the focus shifted to encapsulating the ROS 2 environment in Docker containers to ensure reproducibility across different machines and simplify deployment. A Dockerfile for an Ubuntu 22.04 + ROS 2 Humble base image was designed, including essential development tools (CMake, compilers, Python) and network utilities, and the ROS 2 workspace for the project was integrated into the image. The Docker image was built and tested on the Raspberry Pi, validating that ROS 2 nodes would run correctly inside the container. The container build and run procedures were documented in the GitHub repository, including typical command sequences and troubleshooting notes. At the end of Week 2, the project had a containerized

ROS 2 environment that could be rebuilt and deployed consistently, minimizing configuration drift and simplifying collaboration.

Week 3: First Simulation and IMU Link Prototyping

Week 3 aimed at validating ROS 2 networking through simulation and at establishing the first prototype of the IMU communication pipeline. The `turtlebot3_simulations` package was deployed on the Raspberry Pi, running Gazebo in headless mode to generate virtual sensor data, while RViz2 was executed on a laptop and connected to the Raspberry Pi over the local network, confirming that simulated topics could be visualized remotely. In parallel, an initial Bluetooth link between an ESP32-based IMU module and the Raspberry Pi was established, and raw frames were received via serial-over-Bluetooth. A preliminary ROS 2 node was implemented to parse the IMU frames and publish them as custom messages, validating the basic sensor-to-ROS path. At the same time, CPU saturation and performance limitations were observed on the Raspberry Pi when simulation, sensor parsing and logging were combined, revealing the need for a more powerful core platform.

Week 4: Migration to Jetson Nano and Performance Evaluation

The fourth week was dedicated to adopting the NVIDIA Jetson Nano as the main embedded platform and replicating the existing ROS 2/Docker environment with improved performance. The Jetson firmware was updated and a lightweight desktop environment (Xubuntu 20.04) was installed to reduce baseline resource usage. ROS 2 Humble and Docker were then installed, the *BadKitten* GitHub repository was cloned, and the ROS 2 Docker image was rebuilt for ARM64 with NVIDIA support, preparing the system for future GPU-accelerated components. The TurtleBot3 simulation and ROS 2 nodes were run on the Jetson, revealing smoother performance and greater CPU headroom than on the Raspberry Pi, and the Bluetooth link between the ESP32-IMU and the Jetson was re-established, confirming that sensor data could be streamed reliably to the new platform. By the end of Week 4, the Jetson Nano was fully operational as the primary embedded system, with simulation and IMU data successfully integrated and a clear performance advantage over the Raspberry Pi.

Week 5: IMU Bridge Node and Data Logging

In Week 5 the focus moved to developing a robust ROS 2 bridge node for the IMU and implementing structured logging for later analysis. The `imu_bridge_node` was implemented in ROS 2, responsible for reading Bluetooth serial frames from the ESP32, parsing the values and publishing them as `sensor_msgs/msg/Imu` messages, and was tuned to operate at the target sampling rate of approximately 200 Hz without buffer overruns or parsing errors. CSV-based logging functionality was added to store raw IMU measurements along with time stamps for offline processing, and short tests with the IMU in static and dynamic conditions were conducted to verify that the published data were consistent and stable. The GitHub documentation was updated to include instructions on pairing the IMU and launching the bridge node in the Jetson Docker container. Week 5 thus produced a stable IMU-ROS 2 bridge with high-rate data publication and persistent logging, creating the foundation for the subsequent implementation of the AHRS algorithm.

Week 6: AHRS Implementation and Real-Time Visualization

Week 6 concentrated on implementing a quaternion-based AHRS algorithm for attitude estimation and visualizing the estimated orientation in RViz2. An AHRS algorithm inspired by Madgwick/Mahony filters was implemented, combining gyroscope integration with accelerometer and magnetometer feedback, and was integrated into the `imu_bridge_node` so that both raw and filtered IMU topics (for example, `/imu/raw` and `/imu/filtered`) were published. RViz2 was configured to display a 3D coordinate frame corresponding to the IMU body frame, driven by the filtered orientation quaternion. Experiments with the IMU at rest and under controlled rotations qualitatively validated that the estimated attitude closely matched the physical motion, and filter parameters (gains and complementary factors) were adjusted based on observed behavior and logs to reduce drift and noise. By the end of Week 6, the system was able to provide stable, high-rate attitude estimates in real time, with a consistent visual representation in RViz2, fulfilling a core objective of the internship.

Week 7: Wireless Optimization and Robustness Testing

During Week 7 the wireless communication pipeline between the Jetson and the GUI was optimized and its latency, jitter and robustness were evaluated. A custom Fast DDS XML profile was designed, specifying a restricted UDP port range, unicast locators and tuned discovery parameters, and was deployed in both the Jetson and GUI Docker containers. Topic frequencies and bandwidth were measured using `ros2 topic hz` and `ros2 topic bw`, focusing on the filtered IMU topic, and experiments with induced Wi-Fi interruptions (disconnect/reconnect cycles) were conducted to assess the system's ability to recover communication automatically. ROS 2 bagfiles were recorded during tests to analyze end-to-end timing and message integrity offline. Week 7 demonstrated that the customized Fast DDS configuration significantly reduced latency jitter and enabled robust reconnection after transient network failures, making the wireless pipeline suitable for real-time monitoring.

Week 8: GUI Integration, Demonstration and Documentation

The final week concentrated on integrating the GUI as a ground station, preparing a demonstration of the full system and finalizing documentation and reporting. A GUI-oriented Docker image was built on the laptop, containing ROS 2, RViz2, `rqt` tools and the necessary dependencies, and was configured to subscribe to the filtered IMU topic and to display the 3D orientation of the sensor, as well as simulated robot states when required. Basic control and monitoring features, such as visual indicators of connection status and simple commands for recalibration or node restart, were implemented. An integrated demonstration combining physical IMU movement, Jetson-side AHRS computation, wireless transmission and GUI visualization was prepared and executed. In parallel, documentation in the GitHub repository was consolidated and refined, and the internship report—including figures, tables and structured descriptions of the system—was prepared. A short demonstration video showing the complete system in operation (IMU module, Jetson Nano, wireless ROS 2 pipeline and RViz2 visualization) is available online¹.

7 Conclusions

This internship report has presented the design, implementation and evaluation of an embedded inertial sensing platform based on ROS 2 and NVIDIA Jetson Nano hardware, developed at the SenDiMonoProg Lab, National Central University. The main technical contributions of the work can be grouped into several themes.

First, a containerized ROS 2 environment was created and deployed on embedded hardware, ensuring reproducibility and simplifying system management. The use of Docker images and a structured GitHub repository allows the entire software stack to be rebuilt and reused on different machines with minimal effort.

Second, a functional sensor pipeline was implemented, integrating a physical IMU—via ESP32 and Bluetooth—into ROS 2 through a dedicated bridge node. The system achieves high-rate data streaming (approximately 195 Hz effective) and structured logging suitable for subsequent analysis.

Third, a quaternion-based AHRS algorithm was implemented and tuned, providing real-time attitude estimates with low drift and acceptable noise characteristics for laboratory navigation experiments. This algorithm transforms noisy multi-axis measurements into a coherent orientation signal suitable as an input for control or fusion algorithms.

Fourth, a wireless communication layer was established between the Jetson and a remote GUI container, using a customized Fast DDS profile to achieve low latency and improved jitter behavior over the lab Wi-Fi network. This demonstrates that ROS 2-based systems can be made robust to realistic network constraints when middleware parameters are carefully selected.

Finally, a ground-station GUI was developed using RViz2 and `rqt` tools, enabling intuitive visualization of IMU orientation and simulated robot states, as well as basic monitoring and control functions. Together, these components constitute a coherent experimental platform that connects physical sensing, embedded processing, networked middleware and user-facing visualization.

From an academic and professional perspective, the internship provided valuable experience in modern robotics software engineering, including embedded Linux administration, ROS 2 middleware configuration,

¹Demonstration video of the embedded inertial sensing platform: <https://youtu.be/9ZpddjTI084>

containerization and sensor fusion. The project served as a bridge between theoretical concepts in inertial navigation and their practical realization on real hardware and networks.

Although the current prototype focuses primarily on attitude estimation using a single IMU, it establishes a solid foundation for more advanced multi-sensor fusion and autonomous navigation systems. The platform is now ready to incorporate additional sensors (GNSS, cameras, odometry) and higher-level algorithms (EKF-based fusion, SLAM, navigation stacks), which can build directly on the infrastructure developed during this internship. Future work will therefore concentrate on extending the sensing modalities, refining the fusion algorithms and deploying full autonomy pipelines on top of the embedded framework described in this report.

Acknowledgements

The author would like to express sincere gratitude to all individuals and institutions that made this internship possible. The Taiwan Experience Education Program (TEEP) and National Central University (NCU) provided the opportunity and financial support to carry out a research internship in Taiwan. Prof. Min-Chun Pan, supervisor at the SenDiMonoProg Lab, Mechanical Engineering Department, offered guidance, feedback and trust throughout the project, framing the inertial sensing work within the broader context of the laboratory's research. Mr. Jheng-Ruei Yu, doctoral student and daily mentor during the internship, provided continuous technical advice on inertial systems, ROS 2 and embedded platforms, and patiently helped resolve practical issues. The students and colleagues at the SenDiMonoProg Lab contributed through discussions, questions and informal collaboration, particularly in topics such as sensor calibration and simulation. Finally, the author's home institution and family offered encouragement and support in pursuing this international research experience. Their combined support was essential to the successful completion of both the technical work and the personal learning objectives of the internship.

References

- [1] P. D. Groves, *Principles of GNSS, Inertial, and Multisensor Integrated Navigation Systems*, 2nd ed., Artech House, 2013.
- [2] J. A. Farrell, *Aided Navigation: GPS with High Rate Sensors*, McGraw-Hill, 2008.
- [3] M. H. Sadraey, *Automatic Flight Control Systems*, Lulu Press, 2012.
- [4] R. Mahony, T. Hamel and J.-M. Pflimlin, "Nonlinear Complementary Filters on the Special Orthogonal Group," *IEEE Trans. Automatic Control*, vol. 53, no. 5, pp. 1203–1218, 2008.
- [5] S. O. H. Madgwick, "An efficient orientation filter for inertial and inertial/magnetic sensor arrays," Report x-io Technologies, 2010.
- [6] G. Welch and G. Bishop, "An Introduction to the Kalman Filter," University of North Carolina at Chapel Hill, Tech. Report, 2006.
- [7] Open Robotics, "ROS 2 Documentation," Available online: <https://docs.ros.org/en/>, accessed 2025.
- [8] eProsima, "Fast DDS Documentation," Available online: <https://fast-dds.docs.eprosima.com/>, accessed 2025.
- [9] Docker Inc., "Docker Documentation," Available online: <https://docs.docker.com/>, accessed 2025.
- [10] NVIDIA Corporation, "NVIDIA Jetson Nano Developer Kit User Guide," Available online: <https://developer.nvidia.com/embedded/develop>, accessed 2025.
- [11] S. Macenski et al., "Navigation2: A ROS 2 Navigation Framework," GitHub repository, <https://github.com/ros-planning/navigation2>, accessed 2025.
- [12] S. Macenski, "slam_toolbox: SLAM for the ROS 2 Navigation Stack," GitHub repository, https://github.com/SteveMacenski/slam_toolbox, accessed 2025.

Week	Main focus	Key activities and outcomes
1	Environment setup	Installation of ROS 2 Humble on Raspberry Pi 4; SSH configuration; creation of the <i>BadKitten</i> GitHub repository; first Docker image with ROS 2 and basic nodes.
2	Containerization	Refinement of Docker image; integration of the project workspace; validation of ROS 2 nodes inside containers; initial documentation of installation and usage procedures.
3	Simulation & first IMU tests	First TurtleBot3 simulation on Raspberry Pi; remote RViz2 visualization; initial Bluetooth link between ESP32-IMU and Raspberry Pi; preliminary ROS 2 IMU publisher node.
4	Migration to Jetson Nano	Firmware and OS upgrade of Jetson Nano; installation of Xubuntu/ROS 2/Docker; cloning of GitHub repo; reconstruction of containers; simulation on Jetson with improved performance; re-establishment of IMU connection.
5	IMU bridge and logging	Implementation of <code>imu_bridge_node</code> ; stable reading of IMU frames at ~200 Hz; publication of raw IMU data in ROS 2; first CSV-based logging of sensor streams.
6	Full AHRS and RViz2	Integration of quaternion-based AHRS (Madgwick/Mahony); publication of filtered orientation; configuration of RViz2 for 3D visualization of the IMU frame; calibration and preliminary error assessment.
7	Wireless optimization	Design and deployment of a custom Fast DDS profile; measurement of latency and jitter using ROS tools; robustness tests under Wi-Fi reconnection; use of <code>ros2 bag</code> for end-to-end logging.
8	GUI integration & wrap-up	Construction of the GUI Docker container; ground-station visualization and control; integrated demonstration (IMU-Jetson-GUI-simulation); final documentation and preparation of the internship report and presentation.

Table 1: Weekly overview of internship activities, showing the progression from basic embedded setup and simulation to a fully integrated wireless IMU-ROS 2-GUI system.